基于树莓派与 Sense HAT 模拟器的智能环境监测系统

--架构设计、实时通信与预测分析

本系统采用树莓派桌面操作系统内置的 Sense HAT 模拟器（Sense HAT Emulator）作为数据源提供者[5]。

Sense HAT 模拟器与 API 介绍

Sense HAT 原本是为国际空间站（ISS）的 Astro Pi 教育项目设计的硬件扩展板，集成了陀螺仪、加速度计、磁力计、温度、气压、湿度传感器以及一个 8x8 的 RGB LED 矩阵[7]。为了在没有物理硬件的树莓派 3 上进行系统开发，官方提供了基于 GTK 框架的交互式桌面应用程序 sense-emu-gui 及其配套的 Python 库 sense_emu[8]。

该模拟器在 API 设计上实现了与真实 sense_hat 库的完美镜像。开发人员只需在代码顶部将导入语句 from sense_hat import SenseHat 替换为 from sense_emu import SenseHat，即可无缝迁移所有硬件调用逻辑[9]。在数据采集层面，系统通过高频调用核心 API 来获取环境参数：

温度数据获取：利用 sense.get_temperature()或 sense.get_temperature_from_humidity()接口读取模拟的摄氏温度值[6]。

湿度与气压获取：分别通过 sense.get_humidity()和 sense.get_pressure()方法捕获相对湿度百分比和百帕（hPa）气压值[9]。

仿真环境中的随机误差与抖动注入

值得深入探讨的是，sense_emu 并非仅仅返回静态的标量值。为了最大限度地还原真实物理世界中的传感器行为，模拟器底层算法根据真实传感器的硬件规格人为注入了随机误差（Random Errors）和热噪声（Jitter）[8]。当操作人员在 GTK 图形界面拖动温度、气压或湿度的滑动条（Sliders）时，API 返回的传感器数值并不会发生阶跃式的突变，而是以类似真实传感器热传导速率的渐进方式，伴随着微小的上下浮动，逐渐收敛于目标设定值[8]。开发者必须在软件层面引入移动平均（Moving Average）等数字信号处理算法来消除

高频白噪声

数据持久化(选择 VictoriaMetrics 单节点版)

VictoriaMetrics(VM)单节点版的核心优势在于其极简的运行模式：单个静态二进制文件且无任何外部依赖。在 aarch64 架构下，VM 展现出了卓越的资源利用率，其 CPU 消耗仅为 Prometheus 的 1/1.7，而内存占用则低至其 1/5。

VM 处理生产数据的压缩率比 TimescaleDB 高出 70 倍，平均每个样本仅占用 0.4 至 0.6 字节。

SD 卡写缓冲策略

VictoriaMetrics 引入了 inmemoryDataFlushInterval 标志，该机制允许数据库在内存中暂存摄取的数据，并按设定的时间间隔（例如每小时或每五分钟）才执行一次大规模的批量刷盘。通过利用 LSM-Tree 的顺序写优势，从根本上规避了 SD 卡处理碎片文件时的写放大效应，显著延长了物理寿命。针对树莓派存储介质脆弱的共性问题

小内存管理

通过-memory.allowedPercent 标志将可用内存严格限制在总内存的 40%以内。VM 的内存占用主要由内部序列缓存（tsid 缓存）和倒排索引驱动。下表是 VictoriaMetrics 在树莓派 3B+的推荐配置参数：

标志	推荐值	备注
-inmemoryDataFlushInterval	5m	将 1Hz 写入缓冲至内存，减少对 SD 卡的物理磨损
-memory.allowedPercent	40	为 Flask 后端和 Z-Score 预测算法预留 60%的系统内存
-retentionPeriod	1(month)	限制历史数据规模，保持磁盘 I/O 性能稳定
-search.maxQueryDuration	5s	防止复杂历史趋势查询消耗过长 CPU 时间

考虑到系统需要在 aarch64 架构下稳定运行，且明确涉及了 Z-Score 异常检测和基于

趋势的线性回归预测提出以下选型：

VictoriaMetrics 单节点版。尽管其内存占用略高于 TT，但其优化的 MetricsQL 函数库可以直接在数据库计算预警数据省去大量数据传输从而降低 Python 主进程的计算压力，提高实时告警的响应速度。

系统可靠性

在长期的自动化监测运行中需要自动错误处理。

写入容错：采集循环用 try...except。如果 HTTP 写入请求返回非 2xx 状态码则利用系空闲内存创建一个环形缓冲区，待数据库恢复后再进行批量补推（Backfilling）。

SD 卡保护：启用 log2ram，利用 log2ram 工具日志输出重定向内存关机后持久化到 SD 卡。**禁用交换分区，**防止数据库数据由于物理内存不足被换出（Swap out）到 SD 卡，系统将由于磁盘等待引发告警延迟。

类型安全：参数化查询。时序数据库存在隐式类型锁定。在构造 Line Protocol 字符串时应确保从传感器读取或从用户界面传入的所有变量都经过严格的类型强制转换（Type Casting）防止数值错误

后端架构与实时通信

实现环境监测界面的实时动态更新且无需页面刷新（Page Refresh），是本系统的核心功能要求之一[1]。传统的 HTTP 协议基于请求-响应模型，客户端必须通过短轮询（Short Polling）或长轮询（Long Polling）来获取最新数据。这种模式不仅造成巨大的网络带宽浪费，而且会导致服务器频繁建立和销毁 TCP 连接，极大地消耗树莓派的 CPU 资源[20]。为此，系统采用了基于 WebSocket 协议的全双工模式。

Flask-SocketIO 与异步网络库

系统的后端服务基于 Python 的 Flask 框架构建集成 Flask-SocketIO 扩展[19]。Flask-SocketIO 能够在支持的浏览器中建立持久化连接。

为了在单线程的 Python 环境中实现高并发实时数据流推送，系统舍弃了 Flask 默认的同步开发服务器（Werkzeug），转而采用 Eventlet 异步非阻塞网络库[22]。

后台发射器与事件驱动模型

采集逻辑通过 socketio.start_background_task() 立即孵化工作线程，确保即使没有 Web 浏览器接入，树莓派依然能持续执行数据采集、告警判定、本地存储及物理端提示。

该后台线程按照设定的时间间隔（秒级）执行以下管线操作：

1. 调用 sense_emu API 获取最新的温度、湿度和气压读数。
2. 将清洗后的数据点通过 SQLite 游标持久化至本地数据库。
3. 将包含时间戳和环境数据的 JSON 载荷（Payload），利用 socketio.emit('sensor_update',payload,broadcast=True)广播至所有已连接的 Web 客户端[22]。
4. 调用 socketio.sleep()挂起协程，等待下一次采集周期的到来[25]。

这种事件驱动（Event-Driven）的架构确保了传感器数据的采集、存储与分发被无缝解耦，极大地提升了系统的吞吐效率。

响应式前端可视化与动态双向预警系统

用户与物联网系统进行双向交互。

基于 ThingsBoard 的实时流式渲染架构研究

在物联网应用中，传感器产生的数据流具有高频率、长时序的特点。为了避免从零开始编写底层渲染逻辑，本系统集成了 ThingsBoard 平台可视化引擎。

ThingsBoard 原生支持 WebSocket 协议，能够自动建立与服务器的长连接。当后台传感器产生温湿度或气压的新读数时，平台会通过 WebSocket 实时推送遥测（Telemetry）更新，无需手动解析复杂的 JSON 载荷或编写自定义的事件回调函数 。自动化时间窗口与内存管理

在配置仪表盘组件时，用户可以设定特定的实时时间跨度（例如仅展示最近 60 个数据点或最近 1 小时的温度趋势）。ThingsBoard 的底层逻辑会自动执行类似于数组“推入（push）”与“剔除（shift）”的平衡操作：每当新遥测数据到达，系统将其绘入图表，并自动丢弃超出预设窗口范围的旧数据点 。这种平台级的自动化管理确保了前端内存占用始终保持在恒定水平，使监控系统能够支撑数周甚至数月的连续运行而无需重启浏览器

ThingsBoard 规则引擎与双向交互逻辑

ThingsBoard 通过规则引擎（Rule Engine）和 RPC（远程过程调用）机制，实现全双工双向交互。

规则链驱动的双向事件模型

ThingsBoard 的规则引擎允许通过可视化的拖拽节点构建复杂的数据流转逻辑，替代了传统的 Socket.IO 事件编写：

功能模块	发起/触发端	接收/执行端	描述
遥测更新	传感器设备	仪表盘组件	实时推送最新的温湿度、气压数据
RPC 控制	仪表盘滑块/开关	物理设备	用户调整阈值或开关设备后的下行指令推送
报警触发	规则引擎	全局终端	当遥测数据逸出安全边界时触发的多端广播

用户在前端页面可以通过设置面板中的滑块或输入组件调整阈值。这些操作会触发一个 RPC 调用，直接将新的配置字典推送到服务器并同步至受控设备 。

阈值可配置性与跨终端同步告警

系统内建了一套警报机制（Warning System），旨在当环境条件超出安全范围时发出警报。前端页面提供了一个专门的设置面板（Settings Panel），用户可通过拖动滑块或输入具体数字来调整上述阈值边界。当用户提交新的配置时，前端通过 Socket.IO 发射一个 update_thresholds 事件，将新的配置字典推送至服务器 [22]。

服务器接收后，立即更新驻留在内存中的全局阈值变量，并将其作用于后台数据采集线程的下一次轮询。一旦最新采集的数据逸出用户自定义的边界，系统将触发双管齐下的跨终端告警：

1. **Web 界面视觉告警：**服务器向前端广播 warning_alert 事件，前端通过操控 DOM（Document Object Model），使对应的数据卡片变为高亮红色并伴随闪烁动画，清晰醒目地提示用户 [1]。

2. **物理/模拟器终端告警：**服务器同时调用 Sense HAT 模拟器的 LED 矩阵控制 API，如执行 sense.clear(255,0,0)将 8x8 LED 阵列点亮为纯红色，或通过 sense.show_message("WARNING",text_colour=)滚动播放警告字符串 [9]。这一软硬件联动的机制，确保了无论用户是否在监控屏幕前，都能及时接收到危机信号。

根据系统设计要求，默认的安全区间如下表所示：

环境参数	默认下限阈值	默认上限阈值	物理意义与潜在风险
温度 (Temperature)	$0^{\circ}C$	$40^{\circ}C$	结冰风险或设备严重过热、火灾隐患
湿度(Humidity)	10%	90%	极度干燥可能引发静电，过高湿度导致电路板凝露短路
气压(Pressure)	$970\ \mathrm{hPa}$	$1030\ \mathrm{hPa}$	极端低压可能预示风暴天气，高压可能影响精密仪器标定

复杂时序数据挖掘与高级预测分析模型

针对作业中关于“温度数据分析与预测特征”的严格要求，系统在后端集成了数理统计信号处理。

突发异常波动检测(Spikes and Drops Detection)

环境温度的骤然升高（Spikes）或急速下降（Drops）往往是局部异常事件（如火源靠近、制冷系统崩溃）的最早征兆。然而，由于 sense_emu 模拟器内置了白噪声特性，简单的两点差分法（$T_t - T_{t-1}$）极易因硬件抖动而产生大量假阳性（False Positives）告警[14]。

为了提高检测的鲁棒性，系统采用了**滑动窗口 Z-Score（标准分）异常检测算法**[33]。该算法的核心思想是评估当前数据点偏离近期局部基准线的统计学显著程度。首先，系统在内存中维护一个固定长度（例如$N = 10$）的先进先出（FIFO）温度数据队列。在每个时间步t，计算该时间窗口内温度的局部均值μ与样本标准差σ：

$$\mu = \frac{1}{N}\sum_{i=1}^{N} T_{t-i}$$

$$\sigma = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(T_{t-i} - \mu)^2}$$

当新的温度读数T_t抵达时，计算其对应的 Z-Score：

$$Z = \frac{T_t - \mu}{\sigma}$$

若$|Z|$大于预设的敏感度阈值（通常取$Z > 3.0$，即遵循3σ准则），则系统从统计学角度断定该读数非随机噪声，而是一个具有物理意义的显著突变（Spike 或 Drop）[35]。随后，系统将生成特定的异常事件日志并推送至 Web 仪表盘。

宏观温度趋势识别(Trend Identification)

除了瞬时突变，识别环境中缓慢但持续的升温或降温趋势（Consistent Upward or Downward Trends）对于预防性维护同样至关重要[1]。由于高频采样的时序数据充满了锯齿状的微小波动，系统引入了简单移动平均（Simple Moving Average,SMA）算法作为低通滤波器（Low-pass Filter）来平滑数据[13]。

通过设定较长的时间窗口（如过去 30 个采样点），系统计算连续滑动窗口的平均值序列。如果当前周期的移动平均值严格大于前一周期的移动平均值，并且这种单调递增的关系在连续K个检查点中均成立，系统即可在宏观上确认并标记出一个“持续上升趋势”[37]。这一平滑处理消除了微观噪声的干扰，使得趋势的判定更加平稳和可靠。

线性回归模型的阈值越限时间预测(Predictive Analytics)

当明确识别出温度的持续上升或下降趋势后，系统的最终目标是实施预测性分析（Predictive Features）：估计温度在未来的哪一个具体时间点可能突破用户设定的危险阈值边界[1]。为了在计算效率与模型可解释性之间取得平衡，系统摒弃了计算昂贵且黑盒化的深度学习网络（如 LSTM），转而采用了经典的**普通最小二乘法（Ordinary Least Squares,OLS）单变量线性回归**模型[32]。

单变量线性回归假设时间（自变量X）与温度（因变量Y）之间存在线性依存关系，模型可表示为：

$$Y = \beta_0 + \beta_1 X + \epsilon$$

其中，β_1为斜率（表示温度随时间的变化率），β_0为截距，ϵ为随机误差项。为了使模型聚焦于最新的环境动态，系统从本地 SQLite 数据库中提取最近一段时间（例如过去 5 分钟）的历史数据作为训练样本集合$\{(X_1, Y_1), (X_2, Y_2), \ldots, (X_M, Y_M)\}$[40]。

根据最小二乘法的闭式解（Closed-form solution），最佳拟合直线的斜率与截距可通过以下解析公式直接计算，避免了迭代优化的计算开销[41]：

$$\bar{X} = \frac{1}{M}\sum_{i=1}^{M} X_i, \quad \bar{Y} = \frac{1}{M}\sum_{i=1}^{M} Y_i$$

$$\beta_1 = \frac{\sum_{i=1}^{M}(X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^{M}(X_i - \bar{X})^2}$$

$$\beta_0 = \bar{Y} - \beta_1 \bar{X}$$

在 Python 环境中，上述矩阵运算可借由 scipy.stats.linregress 或纯粹的 numpy 向量化操作以极低的延迟完成[32]。

获取模型参数后，预测时间的推演便转化为一个代数求解过程。假设用户配置的温度上限阈值为Y_{target}（如$40^\circ C$），且当前判定为上升趋势（即斜率$\beta_1 > 0$），则预测触达该阈值的未来时间戳X_{target}为：

$$X_{target} = \frac{Y_{target} - \beta_0}{\beta_1}$$

将求解出的预测时间戳X_{target}减去当前时间$X_{current}$，即可得出距离温度越限的剩余时间区间Δt[43]。系统随后将Δt格式化为易于阅读的倒计时文本（如“预计在 3 分钟 45 秒后温度将超过阈值”），并醒目地展示在 Web 界面的预测分析面板中[39]。这种基于趋势的外推能力，真正赋予了系统防患于未然的智能属性。

Works cited

1. Assignment 1_2026.pdf

2. A Study on the performance of Four Regression Models in Predicting Weather Temperature Based on Python,accessed April 4,2026,https://www.itm-conferences.org/articles/itmconf/pdf/2025/01/itmconf_dai2024_02017.pdf

3. Potential Security Vulnerabilities in Raspberry Pi Devices with Mitigation Strategies-ODU Digital Commons,accessed April 4,2026,https://digitalcommons.odu.edu/cgi/viewcontent.cgi?article=1077&context=covacci-

undergraduateresearch

4. Artificial Intelligence(AI)usage and detection-Massey University,accessed April 4,2026,https://www.massey.ac.nz/study/study-and-assignment-support-and-guides/academic-integrity-student-guide/artificial-intelligence-ai-usage-and-detection/

5. Look Ma,no hardware!Coding the Raspberry Pi in a web emulator|Opensource.com,accessed April 4,2026,https://opensource.com/life/16/9/coding-raspberry-pi-web-emulator

6. Getting started with the Sense HAT-Raspberry Pi Projects,accessed April 4,2026,https://projects.raspberrypi.org/en/projects/getting-started-with-the-sense-hat

7. Sense HAT-Raspberry Pi Documentation,accessed April 4,2026,https://www.raspberrypi.com/documentation/accessories/sense-hat.html

8. Sense Emulator—Sense-Emu 1.0 documentation,accessed April 4,2026,https://sense-emu.readthedocs.io/en/v1.0/sense_emu_gui.html

9. Examples—Sense-Emu 1.0 documentation-Sense HAT Emulator,accessed April 4,2026,https://sense-emu.readthedocs.io/en/v1.0/examples.html

10.Raspberry Pi Sensors:Sense HAT Temperature,Pressure,and Humidity-Technical Articles,accessed April 4,2026,https://control.com/technical-articles/raspberry-pi-sensors-sense-hat-temperature-pressure-and-humidity/

11.Sense-Emu Documentation,accessed April 4,2026,https://sense-emu.readthedocs.io/_/downloads/en/v1.1/pdf/

12.python-sense-emu/docs/sense_emu_gui.rst at master-GitHub,accessed April 4,2026,https://github.com/astro-pi/python-sense-emu/blob/master/docs/sense_emu_gui.rst

13.Moving Average Smoothing for Data Preparation and Time Series Forecasting in Python,accessed April 4,2026,https://machinelearningmastery.com/moving-average-smoothing-for-time-series-forecasting-python/

14.Peak Signal Detection in Real-Time Time-Series Data-GeeksforGeeks,accessed April 4,2026,https://www.geeksforgeeks.org/data-analysis/peak-signal-detection-in-real-time-time-series-data/

15.InfluxDB vs SQLite comparison-PeerSpot,accessed April 4,2026,https://www.peerspot.com/products/comparisons/influxdb_vs_sqlite

16.SQLite Database on a Raspberry Pi-Random Nerd Tutorials,accessed April 4,2026,https://randomnerdtutorials.com/sqlite-database-on-a-raspberry-pi/

17.Developing a Real-time Dashboard with Flask,Postgres,and Socket.IO|TestDriven.io,accessed April 4,2026,https://testdriven.io/blog/flask-postgres-socketio/

18.Creating Flask Real-time Dashboard-python-Stack Overflow,accessed April 4,2026,https://stackoverflow.com/questions/55908693/creating-flask-real-time-dashboard

19.Real-Time Data Visualisation with Plotly and Socket.IO-Atlantbh Sarajevo,accessed April 4,2026,https://www.atlantbh.com/real-time-data-visualisation-with-plotly-and-socket-io/

20.Flask-SocketIO,accessed April 4,2026,https://flask-socketio.readthedocs.io/

21.Real-time Full-Stack Applications with Socket.io-NamasteDev Blogs,accessed April 4,2026,https://namastedev.com/blog/real-time-full-stack-applications-with-socket-io/

22.Getting Started-Flask-SocketIO,accessed April 4,2026,https://flask-socketio.readthedocs.io/en/latest/getting_started.html

23.Deployment-Flask-SocketIO-Read the Docs,accessed April 4,2026,https://flask-socketio.readthedocs.io/en/latest/deployment.html

24.Help making an IoT Real-time Dashboard-DEV Community,accessed April 4,2026,https://dev.to/askbalubabu/help-making-an-iot-real-time-dashboard-ina

25.Background thread how to use socketio push real time database info to client?-GitHub,accessed April 4,2026,https://github.com/miguelgrinberg/Flask-SocketIO/issues/410

26.Use Flask SocketIO to update webpage everytime a local file changes-Stack Overflow,accessed April 4,2026,https://stackoverflow.com/questions/42549367/use-flask-socketio-to-update-webpage-everytime-a-local-file-changes

27.Build a Real-Time IoT Environmental Dashboard with Python and Javascript-George Witt,accessed April 4,2026,https://wittgeo.medium.com/build-a-real-time-iot-environmental-dashboard-with-python-and-javascript-715fc07d0c95

28.Sense HAT Introduction-by Wojciech Nowicki-Medium,accessed April 4,2026,https://medium.com/@wonowicki/sense-hat-introduction-f4160357664f

29.Reading Live Sensor Data Flask&Raspberry Pi,accessed April 4,2026,https://forums.raspberrypi.com/viewtopic.php?t=259214

30.Display Real Time Updates Using Python,Flask and Websocket-YouTube,accessed April 4,2026,https://www.youtube.com/watch?v=dPhr1YCQTU4

31.getting-started-with-the-sense-hat/worksheet.md at master-GitHub,accessed April 4,2026,https://github.com/raspberrypilearning/getting-started-with-the-sense-hat/blob/master/worksheet.md

32.Linear Regression in Python,accessed April 4,2026,https://realpython.com/linear-regression-in-python/

33.How to create a rolling window function to detect spikes in a time-series dataset in python?,accessed April 4,2026,https://stackoverflow.com/questions/66390316/how-to-create-a-rolling-window-function-to-detect-spikes-in-a-time-series-datase

34.A Gentle Introduction to Threshold-Moving for Imbalanced Classification-MachineLearningMastery.com,accessed April 4,2026,https://machinelearningmastery.com/threshold-moving-for-imbalanced-classification/

35.Time Series Pattern Recognition with Air Quality Sensor Data|Towards Data Science,accessed April 4,2026,https://towardsdatascience.com/time-series-pattern-recognition-with-air-quality-sensor-data-4b94710bb290/

36.What is a pythonic method to find a step(or spike)shape in a time series?-Stack Overflow,accessed April 4,2026,https://stackoverflow.com/questions/55761712/what-is-a-pythonic-method-to-find-a-step-or-spike-shape-in-a-time-series

37.What Is a Moving Average?Calculate It in Python-LearnPython.com,accessed April 4,2026,https://learnpython.com/blog/python-moving-average/

38.Python Moving Average Numpy:Tutorial&Examples-Ikigai Labs,accessed April 4,2026,https://www.ikigailabs.io/multivariate-time-series-forecasting-in-python-settings/python-moving-average-numpy

39.Keep it simple,keep it linear:A linear regression model for time series,accessed April 4,2026,https://towardsdatascience.com/keep-it-simple-keep-it-linear-a-linear-regression-model-for-time-series-5dbc83d89fc3/

40.Linear Regression from Scratch using Python and its Time Complexity(Part I)-Medium,accessed April 4,2026,https://medium.com/analytics-vidhya/day-1-3-linear-regression-from-scratch-using-python-and-its-time-complexity-part-i-372026eb58b6

41.Understanding Linear Regression with Python:Calculating the Slope and Intercept Manually|by Prayush Shrestha|Medium,accessed April 4,2026,https://medium.com/@prayushshrestha89/understanding-linear-regression-with-python-calculating-the-slope-and-intercept-manually-15d2a9e918f7

42.Calculating Slopes in Numpy(or Scipy)-python-Stack Overflow,accessed April 4,2026,https://stackoverflow.com/questions/9538525/calculating-slopes-in-numpy-or-scipy

43.Interpreting Slope and Intercept|Python,accessed April 4,2026,https://campus.datacamp.com/courses/introduction-to-linear-modeling-in-python/building-linear-models?ex=5

44.Raspberry Pi–A Friend or Foe?Cyber Physical Security Challenges-Sepio,accessed April 4,2026,https://sepiocyber.com/wp-content/uploads/2020/07/Raspberry-Pi-Risks-Article-E-Book-12-2020.pdf

45.Securing Your Python Web Application:Tips and Tricks-WebClues Infotech-Medium,accessed April 4,2026,https://webcluesinfo.medium.com/securing-your-python-web-application-tips-and-tricks-461dc023aabb

46.Securing Your Flask Applications:Essential Extensions and Best Practices-Qwiet AI,accessed April 4,2026,https://qwiet.ai/appsec-resources/securing-your-flask-applications-essential-extensions-and-best-practices/

47.Flask security issue-Raspberry Pi Forums,accessed April 4,2026,https://forums.raspberrypi.com/viewtopic.php?t=247241

48.Security Considerations—Flask Documentation(3.1.x),accessed April 4,2026,https://flask.palletsprojects.com/en/stable/web-security/

49.IOT-Based Monitoring System with Security Testing-IJIRT,accessed April 4,2026,https://ijirt.org/publishedpaper/IJIRT184351_PAPER.pdf

50.Best practices to protect your Flask applications-Escape.tech,accessed April 4,2026,https://escape.tech/blog/best-practices-protect-flask-applications/

51.Sense HAT API Reference-Read the Docs,accessed April 4,2026,https://sense-hat.readthedocs.io/en/latest/api/

52.Building a Real-Time Client Monitoring System with Flask and Socket.IO|by Boata Andrei,accessed April 4,2026,https://medium.com/@boata.andrei88/building-a-real-time-client-monitoring-system-with-flask-and-socket-io-c023fba5e26c

53.Generative Artificial Intelligence in Assessment Policy|Massey University,accessed April 4,2026,https://www.massey.ac.nz/documents/1445/Generative_Artificial_Intelligence_in_Assessment_Policy.pdf

54.ACADEMIC INTEGRITY POLICY-Massey University,accessed April 4,2026,https://www.massey.ac.nz/documents/2005/Academic_Integrity_Policy.pdf